

Yes. Since you're starting with this **beginner multi-table e-commerce dataset**, the best approach is to practice each SQL concept separately before mixing them.

Below is a structured set of **100 questions – 10 questions for each major SQL logic**.

I am **not giving the answers yet**, so you can solve them yourself.

SQL Beginner Practice – 100 Questions

1. SELECT – Basic Data Retrieval

Practice selecting specific columns and entire tables.

1. Display all customers.
 2. Display only `first_name` and `last_name` from customers.
 3. Display all products.
 4. Display `product_name` and `price` from products.
 5. Display all orders.
 6. Display `order_id`, `customer_id`, and `order_date`.
 7. Display all categories.
 8. Display `first_name`, `email`, and `city` of all customers.
 9. Display `product_name`, `price`, and `stock_quantity`.
 10. Display `order_id`, `order_date`, and `order_status`.
-

2. WHERE – Filtering

Practice filtering rows based on conditions.

1. Find customers who live in Delhi.
2. Find customers who live in Mumbai.
3. Find products with a price greater than ₹10,000.
4. Find products with a price less than ₹5,000.
5. Find products whose stock quantity is greater than 30.
6. Find orders with status `Delivered`.
7. Find orders with status `Cancelled`.
8. Find customers who registered after `2025-02-01`.

9. Find products costing exactly ₹799.
 10. Find orders placed after 2025-05-15 .
-

3. Comparison Operators

Practice:

=
>
<
>=
<=
<>

Questions:

1. Find products costing more than ₹50,000.
 2. Find products costing less than ₹1,000.
 3. Find products costing ₹2,499 or more.
 4. Find products with stock less than 20.
 5. Find products with stock greater than or equal to 40.
 6. Find orders placed on 2025-05-10 .
 7. Find customers who registered before 2025-03-01 .
 8. Find products whose price is not ₹799.
 9. Find orders whose status is not Delivered .
 10. Find customers whose customer_id is greater than 5.
-

4. AND / OR / NOT

Practice combining conditions.

1. Find products costing more than ₹1,000 **AND** having stock greater than 20.
2. Find customers living in Delhi **OR** Mumbai.
3. Find products costing more than ₹50,000 **AND** having stock less than 20.
4. Find orders that are Delivered **AND** were placed after 2025-05-15 .
5. Find customers who are from Delhi **AND** registered after 2025-02-01 .
6. Find products that are cheaper than ₹5,000 **OR** have stock greater than 50.

7. Find orders that are not `Cancelled` .
 8. Find customers who are not from Delhi.
 9. Find products costing between ₹1,000 and ₹10,000 using `AND` .
 10. Find orders that are `Delivered` **OR** `Shipped` .
-

5. IN / NOT IN

Practice filtering against multiple values.

1. Find customers from Delhi, Mumbai, or Pune.
 2. Find customers who are not from Delhi, Mumbai, or Pune.
 3. Find products with category IDs 1, 2, or 3.
 4. Find products whose category ID is not 4 or 5.
 5. Find orders with status `Delivered` or `Shipped` .
 6. Find orders whose status is not `Cancelled` or `Processing` .
 7. Find customers with IDs 1, 3, 5, 7, and 9.
 8. Find products with IDs 1, 5, 10, and 15.
 9. Find orders belonging to customers 1, 2, and 3.
 10. Find products belonging to categories 2, 3, and 5.
-

6. BETWEEN

Practice ranges.

1. Find products priced between ₹500 and ₹5,000.
 2. Find products priced between ₹10,000 and ₹70,000.
 3. Find products with stock between 20 and 50.
 4. Find products with stock between 10 and 30.
 5. Find customers with IDs between 3 and 7.
 6. Find orders with IDs between 103 and 110.
 7. Find orders placed between `2025-05-01` and `2025-05-15` .
 8. Find customers who registered between `2025-02-01` and `2025-03-31` .
 9. Find products priced between ₹1,000 and ₹10,000.
 10. Find orders placed between `2025-05-15` and `2025-06-01` .
-

7. LIKE – Pattern Matching

Practice:

```
LIKE '%...%'
```

```
LIKE '...%'
```

```
LIKE '%...'
```

1. Find customers whose first name starts with **A** .
 2. Find customers whose first name starts with **P** .
 3. Find customers whose first name ends with **a** .
 4. Find customers whose name contains **a** .
 5. Find products containing the word **Phone** .
 6. Find products containing **Book** .
 7. Find products whose name starts with **S** .
 8. Find customers whose last name starts with **M** .
 9. Find customers whose email contains **gmail** .
 10. Find products whose name contains **Wireless** .
-

8. DISTINCT

Practice removing duplicate values.

1. Display all unique customer cities.
 2. Display all unique order statuses.
 3. Display all unique category IDs from products.
 4. Display all unique customer IDs from orders.
 5. Display unique product prices.
 6. Display unique registration dates.
 7. Display unique cities from addresses.
 8. Display unique states from addresses.
 9. Display unique quantities ordered.
 10. Display unique unit prices from order items.
-

9. ORDER BY

Practice sorting.

1. Display products from cheapest to most expensive.
 2. Display products from most expensive to cheapest.
 3. Display customers alphabetically by first name.
 4. Display customers in reverse alphabetical order.
 5. Display products by highest stock first.
 6. Display products by lowest stock first.
 7. Display orders from oldest to newest.
 8. Display orders from newest to oldest.
 9. Display customers by registration date.
 10. Display products sorted by price and then stock quantity.
-

10. LIMIT

Practice restricting results.

1. Find the 5 cheapest products.
 2. Find the 5 most expensive products.
 3. Find the 3 products with the highest stock.
 4. Find the 3 products with the lowest stock.
 5. Find the first 5 customers alphabetically.
 6. Find the last 5 customers based on customer ID.
 7. Find the latest 5 orders.
 8. Find the earliest 5 orders.
 9. Find the 5 most expensive products in category 3.
 10. Find the 3 latest registered customers.
-

11. Aggregate Functions

Practice:

COUNT ()

SUM ()

AVG ()

MIN ()

MAX ()

1. Count the total number of customers.
 2. Count the total number of products.
 3. Count the total number of orders.
 4. Find the highest product price.
 5. Find the lowest product price.
 6. Find the average product price.
 7. Find the total stock across all products.
 8. Find the total number of order items.
 9. Find the highest quantity ordered in a single order item.
 10. Find the average quantity ordered.
-

12. GROUP BY

This is an important one.

1. Count customers in each city.
 2. Count products in each category.
 3. Find the average product price for each category.
 4. Find the highest product price in each category.
 5. Find the lowest product price in each category.
 6. Find total stock for each category.
 7. Count orders for each customer.
 8. Count orders for each order status.
 9. Find total quantity sold for each product.
 10. Find total revenue generated by each product.
-

13. HAVING

Practice filtering groups.

1. Find cities having more than 1 customer.
2. Find categories having more than 2 products.
3. Find categories whose average product price is greater than ₹10,000.
4. Find customers who have placed more than 1 order.
5. Find products whose total quantity sold is greater than 2.
6. Find order statuses having more than 2 orders.
7. Find categories whose total stock is greater than 50.

8. Find customers whose total order quantity is greater than 2.
 9. Find products generating more than ₹5,000 revenue.
 10. Find categories whose maximum product price is greater than ₹50,000.
-

14. INNER JOIN

Now we start working with multiple tables.

1. Display customer names along with their orders.
 2. Display customer names and order dates.
 3. Display customer names and order status.
 4. Display product names with their categories.
 5. Display products along with category names and prices.
 6. Display orders along with customer names.
 7. Display order items with product names.
 8. Display order items with product names and quantities.
 9. Display order items with product names and unit prices.
 10. Display customer name, order ID, product name, and quantity.
-

15. LEFT JOIN

Important for understanding unmatched records.

1. Display all customers and their orders.
 2. Find customers who have never placed an order.
 3. Display all categories and their products.
 4. Find categories that have no products.
 5. Display all products and their order items.
 6. Find products that have never been ordered.
 7. Display all customers and their addresses.
 8. Find customers who don't have an address.
 9. Display all orders and their order items.
 10. Find orders that don't have any order items.
-

16. Multi-Table JOIN

Now combine 3 or more tables.

1. Display customer name, order ID, and order date.
 2. Display customer name, product name, and quantity purchased.
 3. Display customer name, product name, and price.
 4. Display customer name, category name, and product name.
 5. Display order ID, customer name, product name, and quantity.
 6. Display customer name, order date, product name, and unit price.
 7. Display customer city, product name, and quantity purchased.
 8. Display customer name, product name, category name, and price.
 9. Display customer name, order ID, product name, quantity, and unit price.
 10. Display customer name and the total amount of every order.
-

17. Calculations

Practice calculated columns.

1. Display product name and price after adding 10% tax.
2. Display product name and price after a 10% discount.
3. Calculate order-item total:

```
quantity × unit_price
```

4. Display product name and total inventory value:

```
price × stock_quantity
```

5. Calculate the total value of each order.
 6. Calculate revenue generated by each product.
 7. Calculate total revenue for all orders.
 8. Calculate the average order value.
 9. Calculate the discounted price after 20% discount.
 10. Display product name, price, stock, and total inventory value.
-

18. CASE

Practice conditional logic.

1. Categorize products as:

- `Expensive` if price > ₹50,000
- `Affordable` otherwise.

2. Categorize stock:

- `Low` if stock < 20
- `Medium` if stock 20–50
- `High` if stock > 50.

3. Categorize orders:

- `Completed` for Delivered
- `In Progress` for Shipped/Processing
- `Failed` for Cancelled.

4. Display customer city and classify Delhi customers as `Delhi Customer` , others as `Other` .

5. Categorize products based on price:

- Under ₹1,000 → Budget
- ₹1,000–₹10,000 → Mid-range
- Above ₹10,000 → Premium

6. Categorize products based on stock.

7. Categorize orders based on status.

8. Display whether each product is `In Stock` or `Out of Stock` .

9. Display whether an order is `Active` or `Completed` .

10. Create a price category for every product.

19. Subqueries

These are very important before moving to intermediate SQL.

1. Find products that cost more than the average product price.
2. Find products that cost less than the average product price.
3. Find the most expensive product.
4. Find the cheapest product.

5. Find customers who have placed at least one order.
 6. Find customers who have never placed an order.
 7. Find products that have never been ordered.
 8. Find products whose price is greater than the price of `Wireless Mouse`.
 9. Find products belonging to the same category as `iPhone 15`.
 10. Find customers who placed an order after the first order date.
-

20. Real Business Questions

These combine several concepts and are excellent final beginner exercises.

1. **Who are the top 5 customers by total spending?**
 2. **Which product has generated the highest revenue?**
 3. **Which category has generated the highest revenue?**
 4. **Which customers have placed more than one order?**
 5. **Which products have never been ordered?**
 6. **What is the average order value?**
 7. **Which customer has spent the most money?**
 8. **Which city has the highest number of customers?**
 9. **Which category contains the most products?**
 10. **Which product has the highest quantity sold?**
-

Recommended Learning Order

Don't try all 200+ concepts at once. Follow this progression:

LEVEL 1

- |
- |— SELECT
- |— WHERE
- |— Comparison Operators
- |— AND / OR / NOT
- |— IN / NOT IN

- └─ BETWEEN
- └─ LIKE
- └─ DISTINCT

↓

LEVEL 2

- |
- └─ ORDER BY
- └─ LIMIT
- └─ COUNT
- └─ SUM
- └─ AVG
- └─ MIN
- └─ MAX

↓

LEVEL 3

- |
- └─ GROUP BY
- └─ HAVING
- └─ CASE
- └─ Calculations

↓

LEVEL 4

- |
- └─ INNER JOIN
- └─ LEFT JOIN
- └─ 3-table JOIN
- └─ 4-table JOIN

↓

LEVEL 5

- |
- └─ Subqueries
- └─ Correlated thinking
- └─ Business Problems

My recommendation: solve them in this order rather than jumping directly to JOINS. In particular, get very comfortable with **GROUP BY + aggregate functions + HAVING** before moving heavily

into multi-table queries. That combination is the foundation for most practical SQL analysis.